rapport parallele

Lucca de Souza Amodio

March 2026

1 Introduction

2 Profilage de la version séquentielle (Ligne de base)

Avant d'entreprendre toute démarche d'optimisation (vectorisation ou parallélisation), il est impératif d'établir une ligne de base (*baseline*) afin de comprendre le comportement initial de l'algorithme et d'identifier ses goulots d'étranglement.

Pour ce faire, nous avons instrumenté le code séquentiel basique en mesurant le temps d'exécution de ses trois composantes principales à chaque itération : le déplacement des fourmis, l'évaporation des phéromones, et la mise à jour globale. La simulation de référence a été exécutée avec 5000 fourmis, et la nourriture a été trouvée à l'itération 2585.

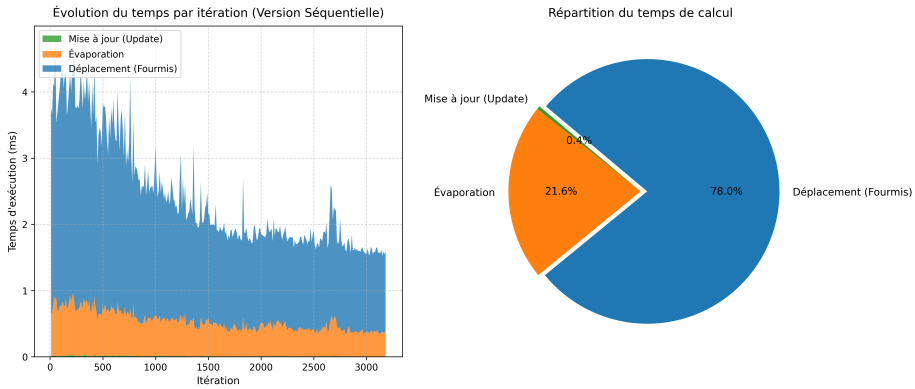


Figure 1: Évolution et répartition du temps d'exécution par itération pour la version séquentielle.

2.1 Analyse de la répartition du temps de calcul

L'analyse de la figure 1 révèle une répartition très inégale de la charge de travail :

- **Le déplacement des fourmis (`Temps_Fourmis`) :** Il constitue le goulot d'étranglement mathématique principal, représentant environ 80% à 82% du temps de calcul d'une itération (environ 3 à 4 ms par itération). Cela s'explique par la complexité de l'heuristique de choix de direction, la génération de nombres aléatoires et la lecture constante des cellules voisines pour 5000 entités indépendantes.

- **L'évaporation (`Temps_Evaporation`) :** Elle occupe la deuxième place (environ 17% à 19% du temps, soit ~ 0.7 ms). Bien qu'il s'agisse d'un parcours complet de la matrice (1024×1024), la simplicité de l'opération (une simple multiplication par un coefficient d'atténuation β) rend cette étape moins coûteuse que la gestion des entités.

- **La mise à jour (`Temps_Update`) :** Cette étape est quasi instantanée (moins de 1%) et consiste principalement en un échange de pointeurs (swap) et l'application des conditions aux limites.

De plus, l'évolution temporelle (graphique de gauche) montre que le temps par itération reste globalement stable tout au long de la simulation, avec de légères variations (allant de 3.6 ms à 4.7 ms au total) dues à la dispersion des fourmis sur la carte et aux accès mémoire associés (cache misses).

2.2 Le coût de l'interface graphique (Affichage)

Un autre aspect crucial révélé par le profilage est l'impact de l'interface graphique (SDL2). Sur le temps total d'exécution du programme qui s'élève à 31.5 secondes, le temps de calcul mathématique pur n'est que de 6.7 secondes. Ainsi, près de 78% du temps global du programme est dédié au rendu visuel (`renderer.display` et `win.blit`).

Cette observation justifie la nécessité de ne pas considerer l'affichage graphique lors des mesures de performance (benchmarks) ultérieures afin de ne pas fausser le calcul de l'accélération (Speedup) lors des optimisations CPU.

3 Optimisation par Vectorisation et Mémoire Partagée (OpenMP)

Afin d'améliorer les performances de la version séquentielle de base, nous avons exploré une optimisation orientée sur l'architecture des données en passant d'un modèle AoS (*Array of Structures*) à un modèle SoA (*Structure of Arrays*). L'objectif principal de cette refonte était de favoriser la vectorisation automatique par le compilateur en alignant les données en mémoire.

3.1 Vectorisation standard (Approche SoA sans OpenMP)

Le passage à l'architecture SoA consistait à séparer les attributs des fourmis en plusieurs tableaux distincts (un tableau pour les positions `pos_ants`, un pour les états `state_ants`, et un pour les graines aléatoires `m_seed`) dans une structure `Colonie`. Mathématiquement et théoriquement, cela permet au compilateur d'utiliser les instructions SIMD (*Single Instruction, Multiple Data*), capables de traiter 4, 8 ou 16 fourmis lors d'un même cycle d'horloge.

Cependant, les mesures de performance ont révélé que le temps d'exécution de cette version vectorisée restait très proche, voire ponctuellement inférieur, à l'ancienne version séquentielle. Cette absence de gain s'explique par deux phénomènes architecturaux majeurs :

1. **La divergence conditionnelle (L'ennemi du SIMD) :**
   La règle d'or du SIMD exige que toutes les entités du vecteur exécutent rigoureusement les mêmes instructions. Or, le comportement d'une fourmi dépend d'une boucle `while (consumed_time < 1.)` et de conditions (`if/else`). Par exemple, la fourmi 0 peut traverser un terrain accidenté (itération unique), tandis que la fourmi 1 itérera trois fois. Face à cette divergence, le compilateur C++ "abandonne" la vectorisation SIMD et se rabat sur une exécution séquentielle.

2. **L'effet pervers du SoA sur le Cache (Cache Thrashing) :**
   La vectorisation ayant échoué, la structure SoA pénalise la hiérarchie mémoire. En approche classique (AoS), les données d'une fourmi (position, état, graine) étaient contiguës en RAM, chargées en un seul voyage (*Cache Hit*). En SoA, le CPU doit effectuer trois requêtes mémoire dispersées pour évaluer une seule fourmi (`pos_ants`, `state_ants`, et `m_seed`). Cet éparpillement génère de nombreux défauts de cache (*Cache Misses*).

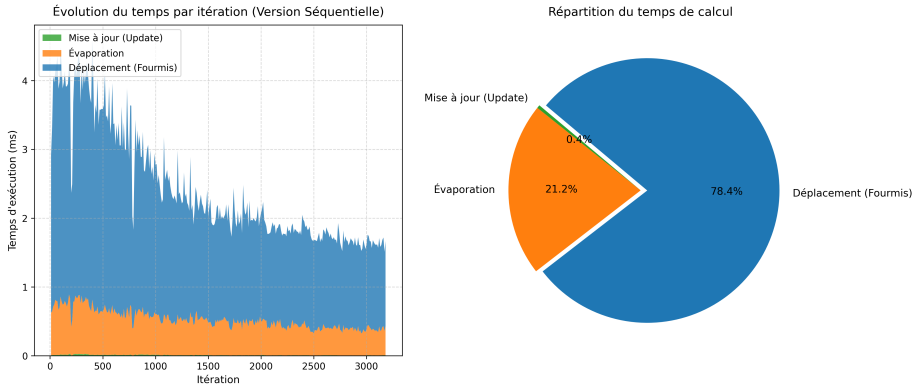


Figure 2: Évolution et répartition du temps d'exécution pour la version vectorisée (SoA), présentant un profil quasi identique à la ligne de base.

Les mesures empiriques (Figure 2) confirment cet échec théorique. L'algorithme reste mathématiquement stable (la nourriture est toujours trouvée à l'itération 2585), mais les performances ne s'améliorent pas. Pire encore, le temps de calcul pur (`Temps_Computation`) subit une légère dégradation, passant de 6717.94 ms pour la version de base à **6785.54 ms** pour la version SoA. Le temps d'exécution total reste quant à lui inchangé (environ 31.5 secondes). Cela prouve que l'overhead lié aux *Cache Misses* a surpassé les éventuelles optimisations mineures du compilateur.

3.2 Vectorisation couplée à OpenMP

Face à l'incapacité du compilateur à vectoriser automatiquement le comportement divergent des fourmis, nous avons introduit une parallélisation explicite à mémoire partagée via OpenMP.

L'idée principale est d'ajouter la directive `#pragma omp parallel for` sur la boucle principale itérant sur les fourmis. Plutôt que de forcer le processeur à exécuter la même instruction sur plusieurs données (SIMD), cette méthode délègue des blocs entiers d'indices de fourmis à des threads (cœurs) différents. Tous les threads accèdent au même espace mémoire partagé (la carte et les tableaux SoA), ce qui évite la duplication massive de données observée précédemment avec MPI.

[Espace réservé pour votre analyse approfondie : Vous pourrez insérer ici les graphiques de Speedup OpenMP, discuter de l'overhead de création des threads, de la gestion des conflits d'écriture sur la grille des phéromones (s'il y en a) et comparer l'accélération obtenue avec celle de la méthode MPI].

4 Parallélisation par échange de carte globale (Première Façon avec MPI et OpenMP)

Dans cette première approche de parallélisation à mémoire distribuée, la stratégie adoptée consiste à dupliquer l'environnement (la carte) sur l'ensemble des processus. Chaque processus ne gère les déplacements que d'un sous-ensemble de fourmis. Lors de la phase de mise à jour, les conflits de phéromones entre processus sont résolus en sélectionnant la valeur maximale locale via une réduction MPI (`MPI_Allreduce` avec `MPI_MAX`). Ensuite, le calcul de l'évaporation est réparti de manière équitable entre les processus, avec une parallélisation interne gérée par OpenMP (en limitant à un cœur par processus, `OMP_NUM_THREADS=1`). La carte globale est finalement reconstruite via la fonction `MPI_Allgather`.

Bien que cette méthode garantisse un excellent équilibrage des tâches, nos mesures mettent en évidence une limitation structurelle majeure liée à l'architecture matérielle et à la taille des données échangées.

4.1 Analyse des performances globales

Comme l'illustre la figure 3, les performances globales se dégradent considérablement à mesure que l'on augmente le nombre de processus. Au lieu d'observer une accélération linéaire, nous constatons un ralentissement sévère : le temps d'exécution passe d'environ 6 secondes (en séquentiel) à plus de 75 secondes avec 12 processus. L'accélération (Speedup) mesurée s'effondre en dessous de 1.

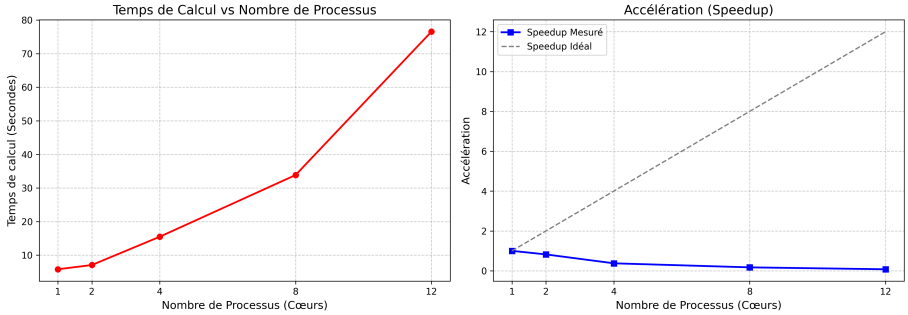


Figure 3: Temps de calcul total et accélération (Speedup) en fonction du nombre de processus.

Ce comportement, bien que contre-intuitif au premier abord, valide les limites théoriques de cette première approche : le gain de temps obtenu par la division du travail est totalement absorbé par le coût des communications inter-processus.

4.2 Identification du goulot d'étranglement : Le coût du réseau

L'analyse de la répartition du temps par itération (Figure 4) localise précisément ce goulot d'étranglement. Les tâches liées au calcul CPU (déplacement des fourmis et calcul local de l'évaporation) représentent une fraction négligeable du temps total.

La quasi-totalité du temps est consommée par `MPI_Allgather` lors de la phase d'évaporation. À chaque itération, les processus doivent s'échanger une grande quantité de données (une matrice de 1024×1024 contenant des paires de `double`, soit environ 16 Mo par processus). Avec 12 processus, le réseau local ou le bus mémoire est saturé par ces transferts massifs. L'approche n'est donc efficace que si le coût de calcul par cellule est colossal par rapport au coût de transfert, ce qui n'est pas le cas ici.

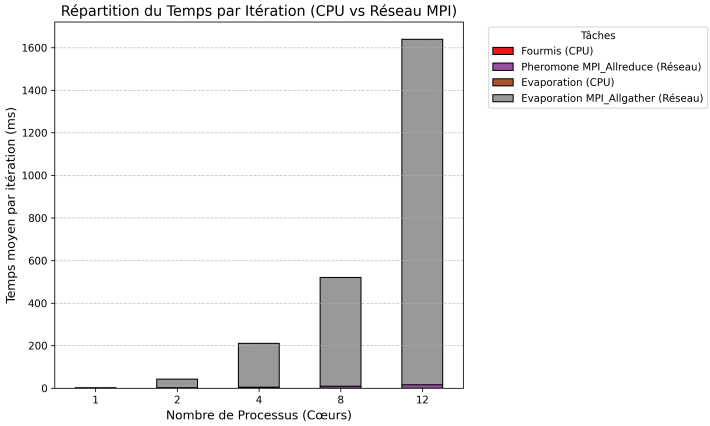


Figure 4: Répartition du temps moyen par itération montrant l'écrasante domination des communications MPI.

4.3 Évaluation du calcul pur : Preuve de la parallélisation algorithmique

Afin de vérifier la validité de notre parallélisation de la logique métier, nous avons isolé les temps de calcul pur (CPU) en excluant les appels bloquants de synchronisation MPI. Le tableau 1 et la figure 5 résument ces résultats.

Nombre de processus (Cœurs)	Temps CPU moyen (ms)	Accélération (Speedup)
1	2.1024	1.00×
2	0.9538	2.20×
4	0.8999	2.34×
8	0.7311	2.88×
12	0.7625	2.76×

Table 1: Résultats de l'accélération en considérant uniquement le temps de calcul pur.

La figure 5 démontre que la répartition des fourmis et le calcul fractionné de l'évaporation fonctionnent. Le temps CPU pur moyen diminue, offrant une accélération atteignant près de 2.88× pour 8 cœurs.

Cependant, la courbe s'aplatit et le temps remonte légèrement avec 12 processus. Cela s'explique par trois facteurs majeurs :

1. **Architecture du processeur (Intel® Core™ i5-13420H) :** Les tests ont été réalisés sur ce processeur qui possède 8 cœurs physiques et 12 threads logiques (utilisant une architecture hybride de P-cores et E-cores). Au-delà de 8 processus, MPI est forcé d'utiliser l'Hyper-threading ou des cœurs à haute efficacité (E-cores). Cela n'apporte pas un gain de puissance

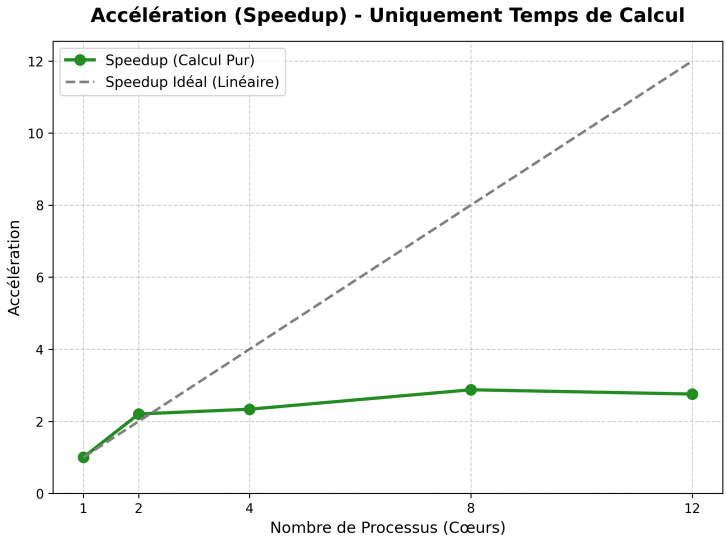


Figure 5: Accélération (Speedup) mesurée uniquement sur les phases de calcul (hors transferts MPI).

de calcul mathématique proportionnel et augmente le temps d'exécution du processus le plus lent, pénalisant l'ensemble de la simulation.

2. **Overhead de parallélisation :** Le temps de calcul de référence (1 processus) n'est que d'environ 2 millisecondes. Diviser une tâche de 2 ms sur 12 processus engendre un surcoût (*overhead*) de gestion des boucles et des structures de données supérieur au temps mathématique gagné.

3. **Goulet d'étranglement mémoire (Memory Bound) :** L'évaporation nécessite le parcours d'une vaste zone mémoire. Avec 12 processus accédant simultanément à la RAM, le goulot n'est plus le CPU, mais la bande passante de la mémoire physique.

4.4 Conclusion sur la Première Façon

L'implémentation de cette méthode met en lumière un compromis classique en Calcul Haute Performance (HPC). Bien que l'équilibrage de charge soit optimal (chaque processus gère un nombre identique de fourmis et de cellules à évaporer), l'immense quantité de données partagées pénalise l'exécution. Conformément aux prédictions de l'énoncé, cette architecture nécessiterait un nombre drastiquement plus élevé de fourmis combiné à une carte beaucoup plus petite pour observer un Speedup global positif.

5 Seconde façon : Décomposition de domaine (Mémoire distribuée)

Dans cette seconde approche de parallélisation, la stratégie adoptée repose sur une décomposition spatiale de l'environnement (décomposition de domaine 1D). Contrairement à la première méthode où chaque processus conservait l'intégralité de la carte en mémoire, ici, la grille globale est divisée en N sous-cartes verticales (où N correspond au nombre de threads). Chaque processus ne gère que les dimensions de sa sous-carte et, par conséquent, uniquement les fourmis qui s'y trouvent physiquement à un instant donné.

5.0.1 Stratégie d'implémentation et Gestion des bords

Le défi principal de cette méthode est la gestion des frontières entre les sous-cartes. Pour qu'une fourmi située sur le bord d'un domaine puisse évaluer correctement les taux de phéromones pour son prochain déplacement (en calculant le maximum local), il est impératif qu'elle ait accès aux informations du domaine voisin.

Pour résoudre ce problème sans provoquer d'erreurs d'accès mémoire (*Segmentation Fault*), nous avons implémenté un système de **cellules fantômes** (*Ghost Cells*). Chaque sous-carte possède une colonne supplémentaire à ses extrémités est et ouest. À la fin de chaque itération, une barrière de synchronisation (`#pragma omp barrier`) est invoquée pour que les processus s'échangent les vraies bordures de leurs cartes et mettent à jour les cellules fantômes de leurs voisins avant la phase d'évaporation.

L'avantage majeur de cette stratégie est la **réduction drastique de l'empreinte mémoire**. Les données échangées entre les processus se limitent strictement aux vecteurs des frontières et aux données des fourmis migrantes, minimisant ainsi le coût de communication global.

5.0.2 Validation du Modèle et Déterminisme Temporel (Bonus technique)

Pour nous assurer que la manipulation des frontières n'altère pas l'intelligence en essaim, nous avons tracé l'évolution de la nourriture rapportée au nid, qui sert d'indice de performance globale.

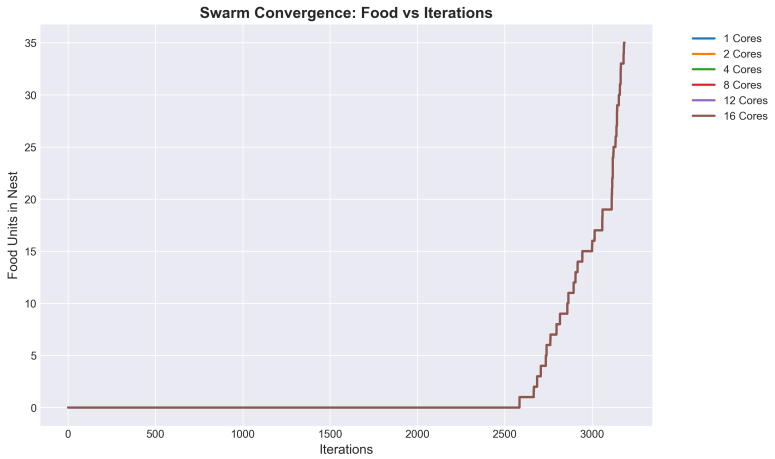


Figure 6: Validation de l'algorithme : Le taux de nourriture rapportée au nid reste strictement identique quel que soit le niveau de parallélisme, prouvant la conservation du comportement macroscopique.

Une complexité supplémentaire est apparue lors du traitement continu du temps (`consumed_time`). Si une fourmi traverse une frontière avec un excédent de temps de mouvement, interrompre brutalement son tour entraînerait une désynchronisation du générateur de nombres pseudo-aléatoires, détruisant le déterminisme de l'essaim.

Pour y remédier, nous avons mis en place une boucle de migration locale (un *sub-loop*). Les threads s'échangent activement les fourmis migrantes jusqu'à ce que **toutes** les fourmis de la carte globale aient consommé exactement leur unité de temps de 1.0. Cela garantit un **déterminisme séquentiel strict** : la convergence de l'algorithme (mesurée par l'arrivée de la nourriture au nid et confirmée par la Figure 6) se produit à l'itération exacte, indépendamment du nombre de cœurs utilisés.

5.0.3 Analyse de l'Équilibrage de Charge (Load Imbalance)

Comme anticipé dans l'énoncé du projet, cette méthode souffre d'un déséquilibre naturel des tâches. Les fourmis ne sont pas réparties uniformément de manière statique. Au début de la simulation, le processus contenant la fourmilière gère la quasi-totalité des agents, tandis que les processus gérant les coins de la carte restent inactifs.

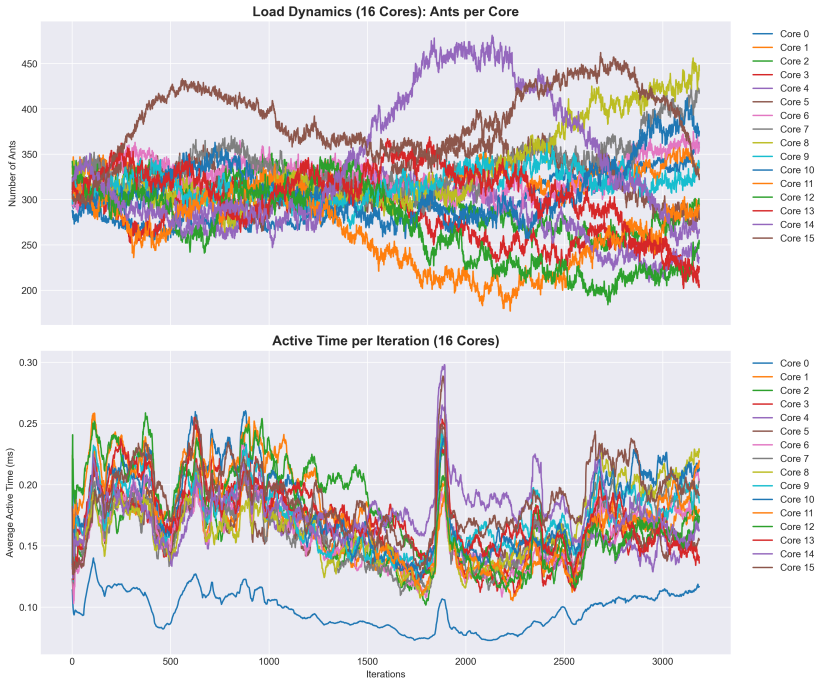
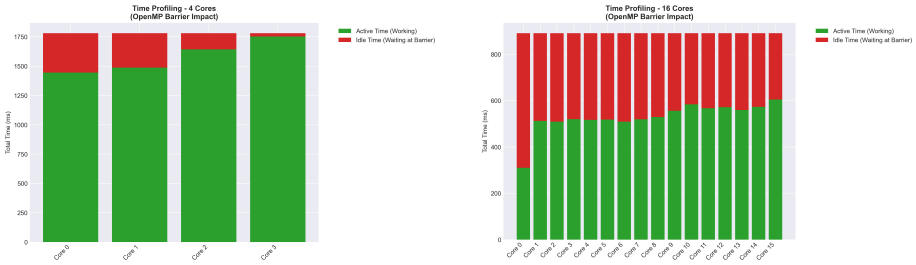


Figure 7: Dynamique de charge sur 16 cœurs. En haut : La migration continue des fourmis entre les sous-cartes. En bas : L'impact direct sur le temps de calcul utile par itération.

La Figure 7 illustre la migration continue des fourmis. Bien qu'avec le temps les fourmis se distribuent sur la carte (comme supposé dans le sujet), cette dynamique spatiale provoque des pics de charge de calcul. Pour visualiser l'impact global de ce goulet d'étranglement, nous avons comparé le profilage temporel sur différentes configurations.



Test sur 4 Cœurs Test sur 16 Cœurs

Figure 8: Évolution du profilage temporel. La partie verte représente le temps de calcul utile (*Active Time*), tandis que la partie rouge représente le temps d'inactivité dans la barrière OpenMP (*Idle Time*).

L'analyse de la Figure 8 confirme ce phénomène géospatial et son aggravation avec l'augmentation du parallélisme. Le temps d'itération total est dicté par le processus le plus lent. On observe clairement que les cœurs gérant les zones entre le nid et la nourriture (par exemple, les cœurs 8 à 15 dans un test à 16 threads) concentrent le temps de calcul utile (barres vertes), forçant les autres cœurs à les attendre passivement dans la barrière OpenMP (accumulation de barres rouges).

5.0.4 Performances : Accélération et Efficacité

En mettant en œuvre cette stratégie, nous avons mesuré le temps d'exécution pur (sans affichage graphique) pour calculer l'accélération (*Speedup*) et l'efficacité parallèle en fonction du nombre de processus.

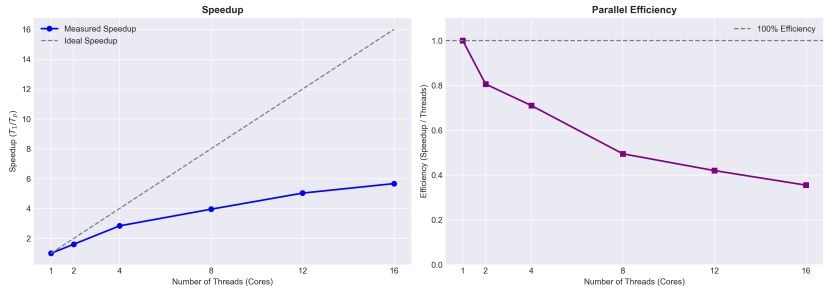


Figure 9: Courbes d'accélération (Speedup) et d'efficacité parallèle en fonction du nombre de processus.

Comme le montre la Figure 9, l'accélération obtenue est évidente, atteignant un Speedup significatif pour 16 processus. Toutefois, la courbe s'éloigne de l'accélération idéale (linéaire), entraînant une baisse progressive de l'efficacité parallèle.

Ce comportement sous-linéaire s'explique parfaitement par la loi d'Amdahl et le *Load Imbalance* démontré précédemment. Les temps d'attente induits par les barrières de synchronisation (`#pragma omp barrier`) pénalisent l'efficacité globale à mesure que l'on augmente le nombre de sous-cartes. Néanmoins, le gain de performance brut et l'économie de mémoire justifient pleinement la pertinence de cette approche pour des domaines de grande taille.